

Linked List Cheatsheet

Patterns, templates, and bug traps for LeetCode linked list problems — Python

Core Mental Model

A linked list is a chain of nodes where each node holds a value and a pointer to the next node. That's it — the whole data structure is `val + next`.

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
```

What this buys you and what it costs you:

Operation	Array	Linked List
Access index <code>i</code>	$O(1)$	$O(n)$ — must walk
Insert/delete at a known position	$O(n)$ — shift everything	$O(1)$ — rewire 2 pointers
Know the length	$O(1)$	$O(n)$
Go backwards	$O(1)$	impossible (singly linked)

Every linked-list problem is designed around those two facts: **you can't jump, and you can't go back**. So the entire toolkit is about (a) getting a pointer to *exactly* the node you need before you need it, and (b) never losing the rest of the chain while you rewire.

The mental discipline that matters most: **draw the boxes and arrows on paper**. Linked list bugs are almost never logic bugs — they're "I overwrote `curr.next` before saving it" bugs, and they're invisible in your head but obvious on paper.

The Universal Setup: Dummy Node

Definition: a dummy node (a.k.a. sentinel) is a fake node you place *before* the real head. `dummy.next` points to the head. You return `dummy.next` at the end.

Why it exists: without it, every insert/delete function needs a special case for "what if the node I'm modifying is the head?" — because modifying the head means changing the variable `head` itself, not a `.next` pointer. The dummy converts the head into an ordinary node that has a predecessor, so the special case disappears.

```
dummy = ListNode(0, head)
prev = dummy

while prev.next:
    if should_delete(prev.next):
        prev.next = prev.next.next    # works even if prev.next was the original head
    else:
        prev = prev.next

return dummy.next    # NOT head — head may have been deleted
```

Rule of thumb: if the problem can possibly modify or remove the first node, use a dummy. That covers remove-nth-from-end, remove duplicates, partition, merge, reverse-sublist, and most others. `return dummy.next`, never `return head`.

The #1 Bug Source: Losing the Rest of the List

When you write `curr.next = something`, the old value of `curr.next` is gone forever — and with it, every node after it. So: **save before you rewire**.

```
nxt = curr.next    # save FIRST
curr.next = prev    # now safe to destroy
curr = nxt          # continue from the saved pointer
```

If you find yourself confused about a rewiring, count how many pointers you're changing. If you're changing k pointers, you need $k+1$ named variables alive at that moment. Reversal changes 1 pointer per step and needs `prev`, `curr`, `nxt` — three names for one rewire. Reverse-in-k-groups changes 3 pointers at the seam and needs 4 names.

Technique 1: Iterative Reversal (the three-pointer dance)

This is *the* linked list primitive. Memorize it cold — it appears inside a dozen other problems.

```
def reverse(head):
    prev = None
    curr = head
    while curr:
        nxt = curr.next    # 1. save the rest of the list
        curr.next = prev   # 2. flip this node's arrow backwards
        prev = curr        # 3. prev advances to the node we just flipped
        curr = nxt         # 4. curr advances into the untouched remainder
    return prev            # curr is None; prev is the last node = new head
```

Traced on 1 → 2 → 3 → None :

step	prev	curr	list state
start	None	1	1→2→3
after 1st loop	1	2	None←1, 2→3
after 2nd loop	2	3	None←1←2, 3
after 3rd loop	3	None	None←1←2←3
return	3	—	3→2→1→None

Two things worth internalizing:

- prev is the head of the reversed portion**, and it's what you return. Returning `head` gives you the tail (which now points to `None`) — a classic silent failure that returns a 1-element list.
- At every moment, the list is split in two: `[reversed part ending at prev] | [untouched part starting at curr]`. That invariant is why the loop is correct.

Recursive version (worth knowing; interviewers ask for it, but it's O(n) stack):

```
def reverse(head):
    if not head or not head.next:
        return head
    new_head = reverse(head.next) # reverse everything after head
    head.next.next = head         # the node after head now points back to head
    head.next = None              # head becomes the new tail
    return new_head               # unchanged all the way up the stack
```

Technique 2: Fast & Slow Pointers (the "runner")

Idea: you can't jump to index `n/2`, but you *can* run one pointer at double speed. When the fast one hits the end, the slow one is halfway. This turns "find a position defined relative to the end/middle" into a single pass with O(1) space.

2a. Find the middle

```
slow = fast = head
while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
# slow is the middle
```

Watch the parity. For `1→2→3→4`, this leaves `slow` at **3** (the second of the two middles). If you need the *first* middle — which you do when splitting a list for merge sort, so the left half isn't empty on a 2-node list — start fast one ahead:

```
slow, fast = head, head.next # now slow lands on 2 for 1→2→3→4
```

The `while fast and fast.next` order matters: check `fast` is non-null *before* dereferencing `fast.next`. Flipping those two crashes on even-length lists.

2b. Nth node from the end

Give fast a head start of `n`, then move both in lockstep. The gap between them stays `n`, so when fast falls off the end, slow is `n` from the end.

```
def remove_nth_from_end(head, n):
    dummy = ListNode(0, head)
    fast = slow = dummy # start both at dummy so slow lands on the PREDECESSOR
    for _ in range(n):
        fast = fast.next
    while fast.next:
        fast = fast.next
        slow = slow.next
    slow.next = slow.next.next # slow is the node before the target
    return dummy.next
```

Note the deliberate off-by-one: to *delete* a node you need its **predecessor**, so both pointers start at `dummy` rather than `head`. This is also why the dummy is mandatory here — deleting the head (`n == length`) otherwise has no predecessor to work with.

2c. Cycle detection (Floyd's tortoise and hare)

If there's a cycle, fast gains 1 position on slow per step, so it eventually laps and lands exactly on slow. If there's no cycle, fast falls off the end.

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    if slow is fast:
        return True
    return False
```

2d. Finding *where* the cycle starts

After the meeting, reset one pointer to `head` and advance both **one step at a time**. They meet at the cycle entrance.

```
def detect_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
    if slow is fast:
        slow = head
        while slow is not fast:
            slow, fast = slow.next, fast.next
        return slow
    return None
```

Why that works (this is the part people memorize without understanding — it's short, so understand it instead). Let `a` = distance from head to the cycle entrance, `b` = distance from the entrance to the meeting point, `L` = cycle length. At the meeting, slow has walked `a + b`, fast has walked `a + b + kL` for some whole number of extra laps `k`. Since fast walks exactly twice as far:

$$2(a + b) = a + b + kL \rightarrow a + b = kL \rightarrow a = kL - b$$

So the distance from head to the entrance (`a`) equals the distance from the meeting point forward to the entrance (`kL - b` is "finish this lap and do `k-1` more"). Two pointers walking at equal speed from those two places arrive together. That's the whole trick.

Technique 3: Merging Two Sorted Lists

The tail-append pattern. Dummy node + a `tail` pointer that always points at the last node of the output.

```
def merge(a, b):
    dummy = ListNode()
    tail = dummy
    while a and b:
        if a.val <= b.val:
            tail.next = a
            a = a.next
        else:
            tail.next = b
            b = b.next
        tail = tail.next
    tail.next = a or b # attach whichever is left over — no loop needed
    return dummy.next
```

Two details that make it clean:

- `tail.next = a or b` — one of them is `None`, and the other is an already-sorted tail you can attach wholesale. You never need to walk it. (`a or b` returns `a` if `a` is truthy, else `b`.)
- Use `<=` not `<` when stability matters (equal elements keep their relative order).

This is also the merge step of **merge sort on a linked list** — the $O(n \log n)$ sort that uses $O(1)$ extra space for pointers (vs. arrays, where merge sort needs $O(n)$ scratch space). Split with the middle-finding pointer, recurse, merge:

```
def sort_list(head):
    if not head or not head.next:
        return head
    slow, fast = head, head.next # first-middle variant
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
    mid, slow.next = slow.next, None # CUT the list in two
    return merge(sort_list(head), sort_list(mid))
```

The `slow.next = None` cut is essential — without it the two halves still point at each other and you recurse forever.

Technique 4: Reversing a Sublist (the anchor pattern)

For "reverse nodes between position `left` and `right`" (LC 92) and "reverse in groups of `k`" (LC 25), the technique is: **grab a pointer to the node just before the section, keep a pointer to the section's original first node (which becomes its last), rewire the two seams.**

```
def reverse_between(head, left, right):
    dummy = ListNode(0, head)
    prev = dummy
    for _ in range(left - 1):
        prev = prev.next      # prev = node before the section (the anchor)

    curr = prev.next          # first node of the section = future tail
    # head-insertion: repeatedly pull the node after curr to the front of the section
    for _ in range(right - left):
        nxt = curr.next
        curr.next = nxt.next  # unlink nxt
        nxt.next = prev.next  # nxt jumps to the front
        prev.next = nxt
    return dummy.next
```

Head-insertion is often easier to get right than "reverse, then reattach," because the anchor `prev` never moves and the two seams are maintained automatically on every iteration.

Reverse in k-groups adds one step: before reversing each group, walk `k` nodes ahead to check a full group exists (if not, leave the remainder alone), then reverse and move the anchor to the group's new tail.

```
def reverse_k_group(head, k):
    dummy = ListNode(0, head)
    group_prev = dummy

    while True:
        # 1. verify k nodes remain
        kth = group_prev
        for _ in range(k):
            kth = kth.next
        if not kth:
            return dummy.next

        group_next = kth.next
        # 2. reverse the group, seeding prev with group_next so the seam closes itself
        prev, curr = group_next, group_prev.next
        while curr is not group_next:
            nxt = curr.next
            curr.next = prev
            prev, curr = curr, nxt

        # 3. reattach the front seam and advance the anchor
        tail = group_prev.next      # old first node = new last node
        group_prev.next = kth      # kth was the old last = new first
        group_prev = tail
```

The clever bit is `prev = group_next` instead of `prev = None`. Standard reversal terminates the reversed chunk with `None`; seeding with `group_next` makes the group's new tail point at the rest of the list automatically, so you only have one seam left to fix by hand.

Technique 5: Two-List Splitting (partition / odd-even)

When you need to rearrange nodes into categories, don't shuffle in place — **build separate chains with separate dummies and concatenate at the end.**

```
def partition(head, x):
    less, more = ListNode(), ListNode()
    lt, mt = less, more
    while head:
        if head.val < x:
            lt.next, lt = head, head
        else:
            mt.next, mt = head, head
        head = head.next
    mt.next = None      # CRITICAL: terminate, or you get a cycle
    lt.next = more.next
    return less.next
```

The `mt.next = None` is the bug everyone hits. The last node in the `more` chain still carries its original `next` pointer, which usually points backwards into the `less` chain → infinite loop. **Whenever you split a list into pieces, explicitly null-terminate every piece.**

Same shape solves odd-even linked list, splitting into halves, and grouping by any predicate.

Technique 6: Reverse-Second-Half Patterns

"Is this a palindrome?" and "reorder the list as first, last, second, second-last, ..." both use the same three-move combination:

1. Find the middle (fast/slow).
2. Reverse the second half.
3. Walk the two halves together.

```
def is_palindrome(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next

    prev = None
    while slow:
        slow.next, prev, slow = prev, slow, slow.next

    while prev:
        if prev.val != head.val:
            return False
        prev, head = prev.next, head.next
    return True
```

Comparing only while `prev` is non-null handles odd lengths for free — the middle element is shared and never needs checking.

Interview note: this mutates the input. Say out loud that you'd restore it if the caller needs the original, or offer the $O(n)$ -space version (copy values into a list, two-pointer it) as the simpler alternative and let them pick.

Technique 7: Copy List with Random Pointer

The problem: deep-copy a list where each node also has a `random` pointer to any node in the list. You can't set `random` on the fly, because its target may not be cloned yet.

Solution A — hashmap (write this one first): two passes with `{original_node: cloned_node}`.

```
def copy_random_list(head):
    if not head: return None
    clone = {}
    curr = head
    while curr:
        clone[curr] = Node(curr.val)
        curr = curr.next
    curr = head
    while curr:
        clone[curr].next = clone.get(curr.next)
        clone[curr].random = clone.get(curr.random)
        curr = curr.next
    return clone[head]
```

`clone.get(...)` rather than `clone[...]` handles `None` targets without a branch.

Solution B — $O(1)$ space interleaving: weave clones into the original list ($A \rightarrow A' \rightarrow B \rightarrow B' \rightarrow \dots$) so each clone sits immediately after its original; then `orig.random.next` is the clone of `orig.random`. Set randoms, then unzip the two lists apart. Mention it as the follow-up optimization.

Bug Table — Check These Before You Submit

Symptom	Almost always the cause
Infinite loop	Split a list without null-terminating a piece, or forgot to advance a pointer inside the loop
Returns only 1 node	Returned <code>head</code> after a reversal instead of <code>prev</code>
Returns the original list unchanged	Returned <code>head</code> after using a dummy; should be <code>dummy.next</code>
<code>AttributeError: 'NoneType' has no attribute 'next'</code>	Dereferenced <code>x.next.next</code> without checking <code>x.next</code> , or checked in the wrong order
Wrong answer only on even-length input	Middle-finding parity — you needed the <i>first</i> middle, not the second
Wrong answer only when modifying the head	No dummy node
Wrong answer only on a 1-node or empty list	Missing <code>if not head</code> or <code>not head.next: return head</code> guard
Nodes disappear from the middle	Overwrote <code>curr.next</code> before saving it

Test every solution against **empty list, one node, two nodes**, and — if there's a k or n — **$k=1$ and $k=length$** . Those five cases catch the overwhelming majority of linked list bugs.

Problem → Technique Map

When you see...	Reach for
Reverse all / part / groups of the list	Three-pointer reversal + anchor <code>prev</code>
Middle, nth-from-end, cycle	Fast & slow pointers
Any insert/delete that could touch the head	Dummy node
Merge, sort, intersection of sorted lists	Dummy + <code>tail</code> append
Palindrome, reorder list, fold	Middle → reverse half → walk together
Partition, odd-even, group by predicate	Two dummies, two chains, null-terminate, concatenate
Deep copy with extra pointers	Hashmap old→new, or interleave for O(1) space
"Intersection of two lists"	Walk both to the end to get lengths, advance the longer by the difference, then lockstep
"Add two numbers"	Dummy + <code>tail</code> , carry variable, <code>while a or b or carry</code>
Need to sort in O(1) space	Merge sort (split at middle, recurse, merge) — not quicksort
Need to go backwards, or need indices	Consider dumping to a Python list first; state the space tradeoff

Pre-Submit Checklist

1. **Did I need a dummy?** If the head can change, yes. Am I returning `dummy.next`?
2. **Every reversal returns** `prev`, not `head`.
3. **Every `.next` dereference is null-guarded**, and the guard order is `while fast` and `fast.next` — never the reverse.
4. **Every split piece is null-terminated.**
5. **Every loop advances something** on every path, including inside `if/else` branches.
6. **Did I save `next` before rewiring?** Count pointers changed; you need one more variable than that.
7. **Traced empty / 1-node / 2-node by hand?** Not "looks fine" — actually walked the pointers.
8. **Space claim honest?** Iterative reversal and fast/slow are O(1). Recursive reversal and recursive merge sort are O(n) and O(log n) stack respectively — say so rather than claiming O(1).

The single highest-leverage habit for this pattern: **before writing any code, draw the before-and-after picture of the pointers for one representative step**, and label which variables you need alive to perform it. Linked list problems are short enough that once the picture is right the code writes itself — and long enough that if the picture is wrong, no amount of staring at the code will save you.